```
# sage des.sage
Plaintext: HAPPINESSISABUTTERFLY
Ciphertext: fffb961a5099b22543c7adf47c8c5ee061d52ff30cc349be
Decrypted Text: HAPPINESSISABUTTERFLY
```

Figure 3: Execution of the SageMath program *des.sage*

mathematical software, can leverage the functionalities offered by PyCryptodome. To install PyCryptodome, simply execute the following command in your terminal: *pip install pycryptodome*. With PyCryptodome installed, you gain access to a comprehensive suite of cryptographic tools, including symmetric-key algorithms such as AES and DES, public-key algorithms like RSA, and hash functions like SHA-3 and BLAKE2.

DES (Data Encryption Standard) is a symmetric-key encryption algorithm used to secure data. It encrypts data by using a fixed-size key to transform the original data into ciphertext and can decrypt it back to the original using the same key. While DES was widely used in the past, it is now considered outdated and less secure due to advances in computational power. However, as the first example, we will discuss the implementation of DES as illustrated in the SageMath program named *des.sage* shown below (line numbers are included for clarity).

```
1.  from Crypto.Cipher import DES
2.  from Crypto.Util.Padding import pad, unpad
3.  key = b"RAINBOWS"
4.  cipher = DES.new(key, DES.MODE_ECB)
5.  plaintext = "HAPPINESSISABUTTERFLY"
6.  print("Plaintext:", plaintext)
7.  plaintext_padded = pad(plaintext.encode(),
    DES.block_size)
8.  ciphertext = cipher.encrypt(plaintext_padded)
9.  print("Ciphertext:", ciphertext.hex( ))
10. decrypted_padded = cipher.decrypt(ciphertext)
11. decrypted_message = unpad(decrypted_padded,
    DES.block_size).decode( )
12. print("Decrypted Text:", decrypted_message)
```

Now, let us go through the code line by line to understand how it works. Line 1 imports the *DES* cipher class from the module *Crypto.Cipher* of the PyCryptodome library, which provides the implementation for the DES encryption algorithm. Line 2 imports two functions, *pad* and *unpad*, from the module *Crypto.Util.Padding*. These functions are used to handle padding for data, which ensures the input is a multiple of the DES block size (8 bytes). Line 3 defines a secret key for the DES algorithm. The *b* before the string indicates it is a byte string, which is required for cryptographic operations. In this case, the key is *"RAINBOWS"*, an 8-character string, which is the exact length required for DES. Line 4 creates a new DES cipher object with the specified key and sets the encryption mode to ECB (Electronic Codebook), one of the simplest modes for block ciphers, where each block is independently encrypted. Line 5 defines the plaintext message to be encrypted: *"HAPPINESSISABUTTERFLY"*. Since DES works on fixed block sizes, the plaintext may need padding. Line 6 prints the original plaintext message to the console for reference. Line 7 encodes the plaintext (converts it into bytes) and then applies padding using the pad function. The padding ensures that the byte length of the plaintext is a multiple of the DES block size (8 bytes). Line 8 encrypts the padded plaintext using the DES cipher and stores the resulting ciphertext in the variable *ciphertext*. Line 9 prints the ciphertext in hexadecimal format for better readability, as the output of the encryption is binary data. Line 10 decrypts the ciphertext using the same DES cipher and stores the result in *decrypted_padded*. The result is still padded to align with the block size. In Line 11, the decrypted message is passed through the function *unpad* to remove the padding, and is then decoded (converted back from bytes to a string) to retrieve the original message. Line 12 prints the decrypted message, which should match the original plaintext, confirming that the encryption and decryption processes worked correctly.

Upon execution, the program *des.sage* generates the output shown in Figure 3. But how does DES work? We will explore this in detail in the next article in this series.

As we wrap up this article, we have broadened our understanding of key cybersecurity terminology. We explored Rail Fence encryption and introduced the concept of symmetric-key encryption. However, there is still much to explore in the realm of symmetric-key cryptography. In the next article, we will continue our discussion by exploring the working of the Data Encryption Standard (DES) and other key developments in this area. END

By: Dr Deepu Benson

The author is assistant professor at Chinmaya Vishwa Vidyapeeth (deemed to be university), Ernakulam, Kerala. He is a free software enthusiast and is interested in theoretical computer science.